

0.106. Une lettre qui tire — la boule de feu

Le 0.105.1, et une deuxième arme : le bouton B lance une boule de feu. Plus lente qu'un tir, elle clignote « O », « # », « O »... comme une flamme, et casse le X qu'elle touche.

1. Ce qu'il faut comprendre

Une deuxième arme : la boule de feu. Le bouton **A** tire toujours ses rafales de « . » (le 0.105.1). Le bouton **B** lance une **boule de feu** : plus grosse, plus lente, et elle **clignote**, comme une flamme qui danse.

Ses variables : comme un tir, une place (`fx`, `fy`), un sens (`fdx`, `fdy`), et `feu`, 1 quand elle vole. **Une seule à la fois :** `bouton(B) && feu == 0`.

Ce qui est nouveau ici (1) : l'effet de flamme. La variable `flamme` dit quel dessin montrer : 0, un « O » ; 1, un « # ». La fonction `dessinerFeu()` dessine la boule, **puis change** `flamme` : `flamme = 1 - flamme`. $1 - 0 = 1$, et $1 - 1 = 0$: elle passe de l'un à l'autre à chaque appel. À chaque pas, la boule a donc l'autre dessin : O, #, O, #...

Ce qui est nouveau ici (2) : une autre vitesse. Elle avance avec son propre `chaque(200)` : un pas toutes les 200 ms, **deux fois moins vite** que les tirs (100 ms). Deux rythmes dans le même programme, sans se gêner (le 0.77).

Le pas de la boule : le même plan que pour un tir (le 0.105). Hors de l'écran, elle disparaît ; sur un X, elle le casse et s'éteint ; sur autre chose, elle s'éteint ; sur une case vide, elle y va, avec l'autre dessin.

Déroulons : A en (2, 8), regard à droite. Bouton B : la boule part en (3, 8), dessin « O », `flamme` passe à 1. 200 ms : (4, 8), « # », `flamme` = 0. 200 ms : (5, 8), « O »... Arrivée (12, 8) : un X, cassé.

Essaie : fais-la plus lente encore, `chaque(400)`, pour bien voir le clignotement. Ou change ses dessins : « ! » et « : ».

Les fonctions de cette leçon — et le tuto de chacune

- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <chaque>` → [0.29.3. La fonction chaque\(\) — toutes les n millisecondes](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)
- `#include <lire>` → [0.68.4. La fonction lire\(\) — lire une case](#)
- `#include <effacer>` → [0.16.3. La fonction effacer\(\) — effacer des cases](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1488 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :
démarrage 91 o ← la console, à l'allumage — la mise en route de la console

```

chaque_minuteur() 124 o ← main() – une fonction fournie par le langage
dessinerFeu()      54 o ← main() – ta fonction
main()             921 o ← démarrage – ta fonction
VBlank             8 o  ← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage      28 o ← image() – attend l'image suivante
AttendreVBlank     16 o ← démarrage – attend que l'écran soit libre
AttendreAcces      12 o ← EffacerCases, EcrireTexte – attend de pouvoir écrire une case
AdresseCase        10 o ← texte(), poser(), lire(), effacer() – trouve l'adresse d'une case de l'écran
EffacerCases       12 o ← effacer() – efface des cases
EcrireTexte        10 o ← texte() – écrit un texte, lettre par lettre
CopierTuiles       67 o ← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte       14 o ← démarrage – vide les deux cartes de l'écran
LireManette        37 o ← bouton() – lit les huit boutons
Tuiles             80 o ← CopierTuiles – 5 lettres sur 43
"0"                1 o  ← texte() – un texte gravé
"#"                1 o  ← texte() – un texte gravé
"."                1 o  ← texte() – un texte gravé
"."                1 o  ← texte() – un texte gravé

```

Lettres gravées : AOX.#

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, DonneesTransfert, Notes

Fonctions du langage appelées :

```

texte()           x 4   104 o sur place, dans dessinerFeu(), main() + AdresseCase (10 o), EcrireTexte (10 o)
effacer()         x 5    91 o sur place, dans main() + AdresseCase (10 o), EffacerCases (12 o)
lire()            x 7    77 o sur place, dans main() + AdresseCase (10 o)
poser()           x 3    38 o sur place, dans main() + AdresseCase (10 o)
bouton()          x 6    30 o sur place, dans main() + LireManette (37 o)
image()           x 1     3 o sur place, dans main() + AttendreImage (28 o)
images()          x 1     2 o sur place, dans chaque_minuteur()

```

Tes fonctions appelées :

```

chaque_minuteur() x 4    43 o sur place, dans main()
dessinerFeu()     x 2     6 o sur place, dans main()

```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```

// ---- UNE LETTRE QUI TIRE : la boule de feu ----
// Le 0.105.1, et une deuxième arme : le bouton B lance une BOULE DE FEU.
// Elle est plus lente qu'un tir, elle CLIGNOTE (« 0 », puis « # », puis « 0 »...)
// comme une flamme, et elle casse le X qu'elle touche.
// ALPHABET[23] : le X.
// ALPHABET[23] : le X.

```

```

#include <texte>      // écrit un texte à l'écran
#include <poser>      // pose une tuile sur une case du fond
#include <ALPHABET>   // les lettres de la police : ALPHABET[0] est le A
#include <chaque>     // répond 1 toutes les n millisecondes
#include <bouton>     // lit un bouton de la manette
#include <lire>       // lit la tuile posée sur une case
#include <effacer>    // efface des cases, ou tout le fond

```

2. UNE VARIABLE — AX

```
uint8_t ax = 2;           // le A : sa colonne..
```

3. UNE VARIABLE — AY

```
uint8_t ay = 8;           // ..et sa ligne
```

4. UNE VARIABLE — DX

```
uint8_t dx = 1;           // le sens où regarde le A
```

5. UNE VARIABLE — DY

```
uint8_t dy = 0;
```

6. UNE VARIABLE — NX

```
uint8_t nx = 0;           // une case d'ARRIVÉE, calculée avant de bouger
```

7. UNE VARIABLE — NY

```
uint8_t ny = 0;
```

8. UN TABLEAU — TX

```
uint8_t tx[3];           // les trois tirs : leurs colonnes...
```

9. UN TABLEAU — TY

```
uint8_t ty[3];           // ...leurs lignes...
```

10. UN TABLEAU — TDX

```
uint8_t tdx[3];           // ...et leurs sens
```

11. UN TABLEAU — TDY

```
uint8_t tdy[3];
```

12. UN TABLEAU — VOLE

```
uint8_t vole[3];           // vole[i] = 1 : le tir n° i est en vol
```

13. UNE VARIABLE — LANCE

```
uint8_t lance = 0;
```

14. UNE VARIABLE — FX

```
uint8_t fx = 0;           // NOUVEAU : la boule de feu : sa colonne...
```

15. UNE VARIABLE — FY

```
uint8_t fy = 0;           // ...sa ligne...
```

16. UNE VARIABLE — FDX

```
uint8_t fdx = 0;           // ...son sens...
```

17. UNE VARIABLE — FDY

```
uint8_t fdy = 0;
```

18. UNE VARIABLE — FEU

```
uint8_t feu = 0;           // ...1 si elle vole, 0 sinon (une seule à la fois)...
```

19. UNE VARIABLE — FLAMME

```
uint8_t flamme = 0;           // ...et son dessin : 0 = « 0 », 1 = « # »

// ---- NOUVEAU : dessiner la boule de feu, et changer son dessin ----
// Une fois « 0 », la fois suivante « # », puis « 0 »... : elle clignote.
// flamme = 1 - flamme : 1 - 0 = 1, et 1 - 1 = 0. Elle passe de l'un à l'autre.
```

20. UNE FONCTION — DESSINERFEU()

```
void dessinerFeu() {
    if (flamme == 0) {
        texte(fx, fy, "0");
    } else {
        texte(fx, fy, "#");
    }
    flamme = 1 - flamme;
}
```

21. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    for (uint8_t l = 4; l < 13; l++) {    // le mur : 9 X en colonne 12 (le 0.101)
        poser(12, l, ALPHABET[23]);
    }
    poser(ax, ay, ALPHABET[0]);

    while (true) {
        image();

        // ---- le A bouge et retient où il regarde ----
        if (chaque(150)) {
            nx = ax;
            ny = ay;
            if (bouton(DROITE)) {
                nx = ax + 1;
                dx = 1;
                dy = 0;
            } else if (bouton(GAUCHE)) {
                nx = ax - 1;
                dx = -1;
                dy = 0;
            } else if (bouton(BAS)) {
                ny = ay + 1;
                dx = 0;
                dy = 1;
            } else if (bouton(HAUT)) {
                ny = ay - 1;
                dx = 0;
                dy = -1;
            }
            if (nx < 20 && ny < 17) {
                if (lire(nx, ny) == 0) {
                    effacer(ax, ay, 1);
                    ax = nx;
                    ay = ny;
                    poser(ax, ay, ALPHABET[0]);
                }
            }
        }

        // ---- le bouton A lance un tir, au plus toutes les 250 ms ----
        if (chaque(250)) {
            if (bouton(A)) {
                nx = ax + dx;
                ny = ay + dy;
                // la case de départ doit être dans l'écran ET vide (le 0.101)
                if (nx < 20 && ny < 17 && lire(nx, ny) == 0) {
                    lance = 0;
                    for (uint8_t i = 0; i < 3; i++) {
                        if (lance == 0 && vole[i] == 0) {
                            vole[i] = 1;
                            tx[i] = nx;
                            ty[i] = ny;
                            tdx[i] = dx;
                            tdy[i] = dy;
                            texte(tx[i], ty[i], ".");
                            lance = 1;
                        }
                    }
                }
            }
        }
    }

    // ---- chaque tir avance, et regarde sa case d'arrivée (le 0.105) ----
```

```

if (chaque(100)) {
    for (uint8_t i = 0; i < 3; i++) {
        if (vole[i] == 1) {
            nx = tx[i] + tdx[i];
            ny = ty[i] + tdy[i];
            effacer(tx[i], ty[i], 1);
            if (nx >= 20 || ny >= 17) {
                vole[i] = 0;          // 1. hors de l'écran
            } else if (lire(nx, ny) == ALPHABET[23]) {
                effacer(nx, ny, 1);    // 2. un X : le tir le casse (le 0.105)...
                vole[i] = 0;          // ...et disparaît
            } else if (lire(nx, ny) != 0) {
                vole[i] = 0;          // 3. un autre tir : il s'écrase
            } else {
                tx[i] = nx;           // 4. une case vide : il y va
                ty[i] = ny;
                texte(tx[i], ty[i], ".");
            }
        }
    }
}
// ---- NOUVEAU : le bouton B lance la boule de feu, dans le sens du regard ----
// Une seule à la fois (feu == 0), sur une case vide à côté du A.
if (bouton(B) && feu == 0) {
    nx = ax + dx;
    ny = ay + dy;
    if (nx < 20 && ny < 17 && lire(nx, ny) == 0) {
        feu = 1;
        fx = nx;
        fy = ny;
        fdx = dx;                      // elle recopie le sens du A, comme un tir
        fdy = dy;
        dessinerFeu();
    }
}

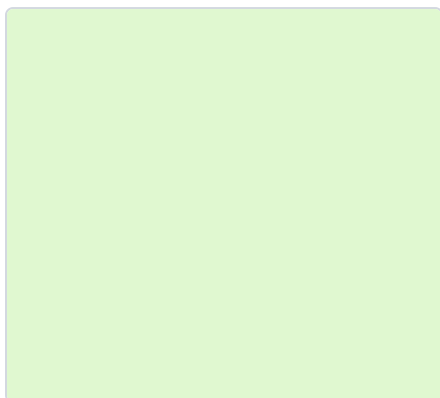
// ---- NOUVEAU : la boule avance toutes les 200 ms : deux fois moins vite qu'un tir ----
if (chaque(200)) {
    if (feu == 1) {
        nx = fx + fdx;
        ny = fy + fdy;
        effacer(fx, fy, 1);           // elle quitte sa case
        if (nx >= 20 || ny >= 17) {
            feu = 0;                  // 1. hors de l'écran : elle disparaît
        } else if (lire(nx, ny) == ALPHABET[23]) {
            effacer(nx, ny, 1);       // 2. un X : elle le casse...
            feu = 0;                  // ...et s'éteint
        } else if (lire(nx, ny) != 0) {
            feu = 0;                  // 3. autre chose : elle s'éteint
        } else {
            fx = nx;                  // 4. une case vide : elle y va...
            fy = ny;
            dessinerFeu();             // ...avec l'autre dessin : elle clignote
        }
    }
}
}
}
}

```

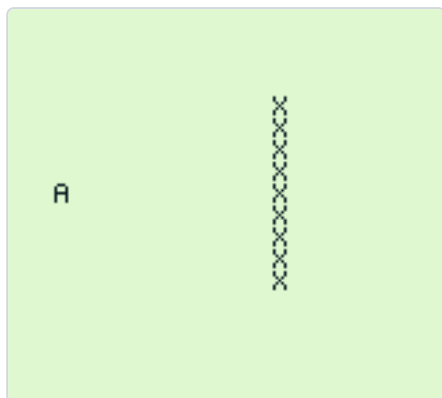
3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est

la vérité sur ce programme.



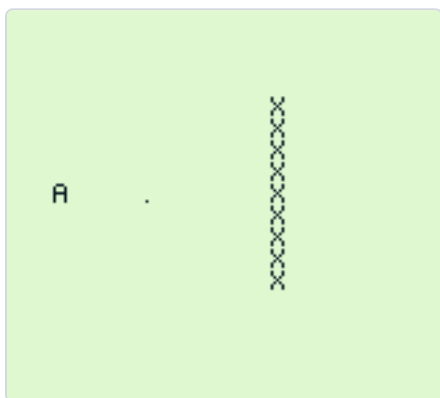
la toute première image — la console vient de démarrer — image 1



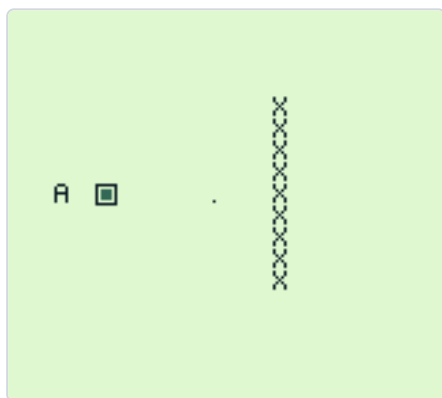
un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

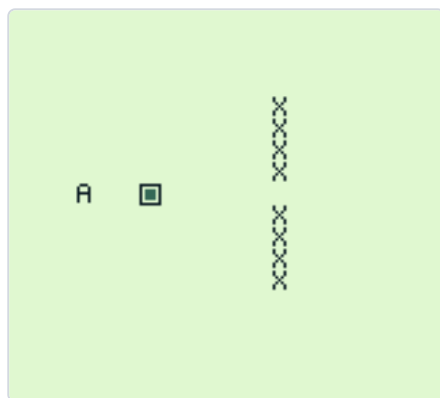
Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



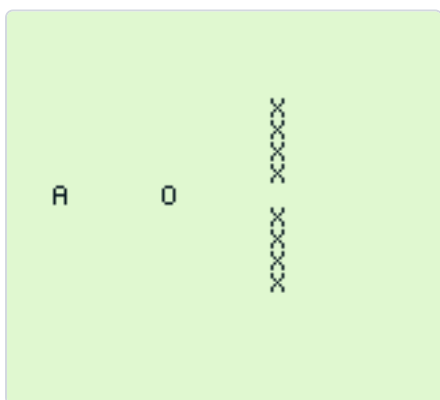
A — l'écran a changé



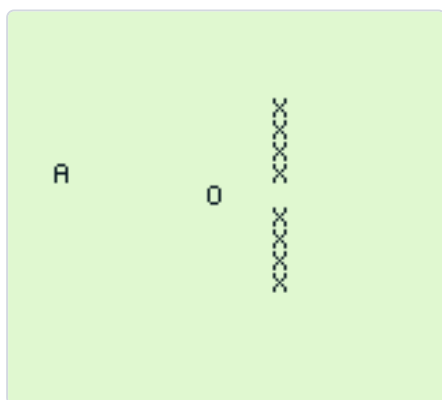
B — l'écran a changé



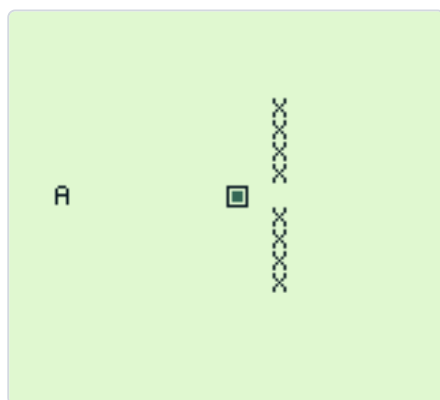
DROITE — l'écran a changé



GAUCHE — l'écran a changé



HAUT — l'écran a changé



BAS — l'écran a changé

5. Ce qu'on doit voir

Le bouton B lance une boule de feu qui avance deux fois moins vite qu'un tir en clignotant « O » / « # », et casse le X qu'elle touche ; A tire toujours des rafales.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ la boule clignote : on voit « O » et « # » sur sa route
- ✓ elle avance d'une case à la fois, jusqu'au mur (colonnes 4 à 11)
- ✓ elle casse le X de la ligne 8, et s'éteint

Ensuite : **0.107. Une lettre qui tire — la boule de feu explose** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++